FairHold: A Refundable Payment Escrow DApp

Maxine Koh Xin Yi
G2604856A
SC6113 -- Development of Decentralised Applications
Trimester 1
Individual Assignment: Financial Decentralised Application

1. Introduction and Problem Statement

Small freelance and peer-to-peer service arrangements often take place between participants with limited prior trust. Payment before delivery exposes a buyer to non-performance, while payment after delivery exposes a seller to non-payment. A conventional escrow service can reduce these risks but introduces intermediary custody and additional institutional trust assumptions.

FairHold investigates whether a rule-based Ethereum smart contract can act as an inspectable custodian for a specified deposit. A buyer deposits test ETH for a designated seller and may later release the funds. If the escrow remains unsettled after an agreed deadline, the buyer can request a refund. The contract records settlement state, while the application presents the actions through a browser-based interface.

The design does not verify whether an off-chain service has been performed. Accordingly, it should not be represented as a neutral or dispute-resolving escrow: the buyer retains substantial authority over settlement.

2. Project Objectives

- Implement a Solidity escrow containing separate buyer and seller addresses, deposited amount, refund deadline and settlement status.
- Integrate MetaMask with a web interface that supports signed transactions on Ethereum Sepolia.
- Use Flask for application delivery, configuration retrieval, input validation and optional read-only receipt verification.
- Display transaction confirmation, escrow state and on-chain event history with block-explorer links.
- Evaluate successful payment and refund scenarios, invalid inputs and relevant security limitations.

3. Scope and Evidence

The implementation is a proof of concept using Sepolia test ETH rather than real funds. Recorded application screenshots demonstrate one escrow released to a seller and another refunded to its buyer. A separate Codespaces terminal screenshot records seven successful Flask unit tests. Direct unauthorized-wallet contract testing, independent block-explorer receipt verification and formal contract auditing were not completed as part of the observed testing.

4. System Architecture

FairHold separates web delivery, off-chain validation, wallet authorization and on-chain settlement. Flask serves the browser interface and validates submitted parameters for usability, but the Solidity contract—not Flask—holds escrowed ETH and enforces transfer conditions. MetaMask signs transactions using the user's selected wallet, and ethers.js v6 mediates contract calls and event queries.

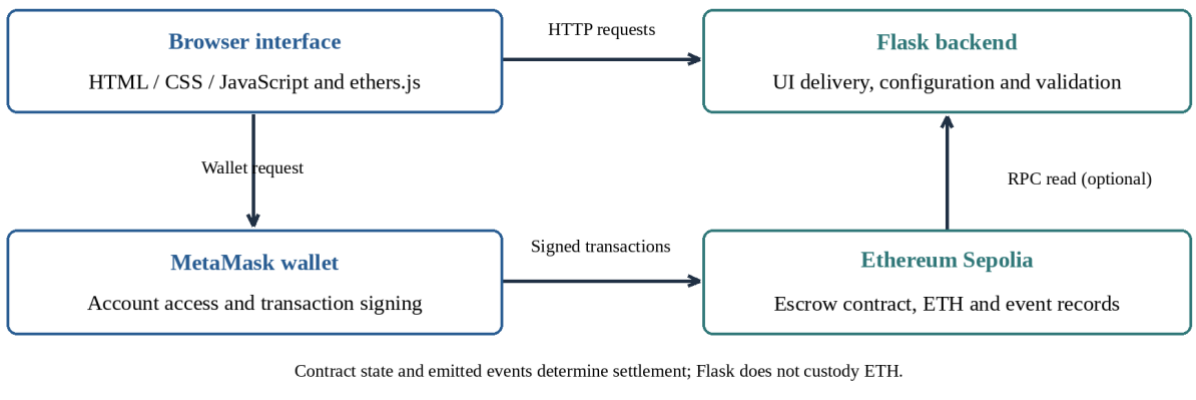


Figure 1. FairHold system architecture and communication boundaries.

The application invokes Flask through HTTP requests for public configuration and a pre-transaction validation check. Signed state-changing transactions are submitted through MetaMask to Sepolia. The browser obtains escrow details and events from the blockchain. An optional Flask endpoint can independently inspect transaction receipts if a Sepolia RPC URL is configured.

5. Technologies Used

Component	Implementation and purpose
Ethereum Sepolia and Solidity 0.8.24	Testnet execution, ETH custody, settlement logic, immutable transaction records.
MetaMask and ethers.js v6	Wallet account access, transaction signing, contract interaction and event retrieval.
Flask (Python)	HTTP serving, public configuration, validation and optional RPC receipt inspection.
HTML, CSS and JavaScript	User interface, account display, escrow forms, alerts and transaction history.
Remix IDE and GitHub	Solidity compilation and deployment; source-control and testing-evidence repository.

6. Rationale for Blockchain Use

A public smart contract provides a shared record of deposits and settlement events that neither party can privately rewrite. Contract execution makes the written settlement rules inspectable and reduces reliance on a third party to hold funds. Nevertheless, blockchain does not establish whether off-chain work was delivered. The fairness of the arrangement depends on the chosen authorization and dispute-resolution rules, not solely on ledger transparency.

7. Smart Contract Design

The FairHoldEscrow contract stores an Escrow structure containing buyer, seller, amount in wei, a Unix timestamp (refundAfter) and a Status enum. Escrow IDs begin at 1, and address-to-ID mappings allow participants to retrieve records associated with their wallets. The contract emits EscrowCreated, EscrowReleased and EscrowRefunded events.

Function	Role and enforcement
createEscrow(seller, refundAfter)	Payable deposit; validates nonzero value, distinct buyer and seller, and a bounded future deadline.
releasePayment(escrowId)	Permits only the buyer to release a funded escrow; transfers the amount to the seller.
refundPayment(escrowId)	Permits only the buyer to reclaim a still-funded escrow after its deadline.
getEscrow(escrowId)	Reads buyer, seller, amount, deadline and status.
getUserEscrows(participant)	Returns escrow IDs involving the specified wallet address.

7.1 Escrow Lifecycle

On successful creation, status is Funded. Exactly one settlement transition may follow: Funded to Released, or Funded to Refunded. Both Released and Refunded are terminal states. No automatic refund occurs when time passes: an authorized buyer must submit a transaction to invoke the refund function.

7.2 Security Controls

- Authorization: releasePayment and refundPayment require msg.sender to equal the escrow buyer. A hidden frontend button is not sufficient protection without this Solidity check.
- State integrity: settlement requires a Funded status, preventing normal repeat settlement of an already released or refunded escrow.
- Checks-effects-interactions: the contract updates status before transferring ETH using a low-level call; if the transfer fails, the transaction reverts.

- Input constraints: createEscrow requires a positive deposit, a seller other than the buyer or zero address, and a refund deadline within an allowed future interval.
- Custody: the contract receives ETH through the payable deposit function and rejects unsupported direct ETH transfers.

7.3 Residual Risks

The rules are deliberately buyer-biased: an honest seller may deliver work but still face non-release followed by a buyer refund after expiry. Recipient contracts may also reject incoming ETH. The contract has not undergone a third-party security audit or systematic fuzz testing and must not be used with real funds.

8. Frontend and Application Implementation

The web application presents a wallet connection control, seller address field, ETH deposit amount, refund deadline, escrow list and on-chain history. After wallet connection, the browser checks for Sepolia (chain ID 11155111), constructs an ethers.js contract instance and submits payable transactions after Flask pre-validation. It waits for the mined receipt, checks successful execution, refreshes escrow state and reports meaningful errors.

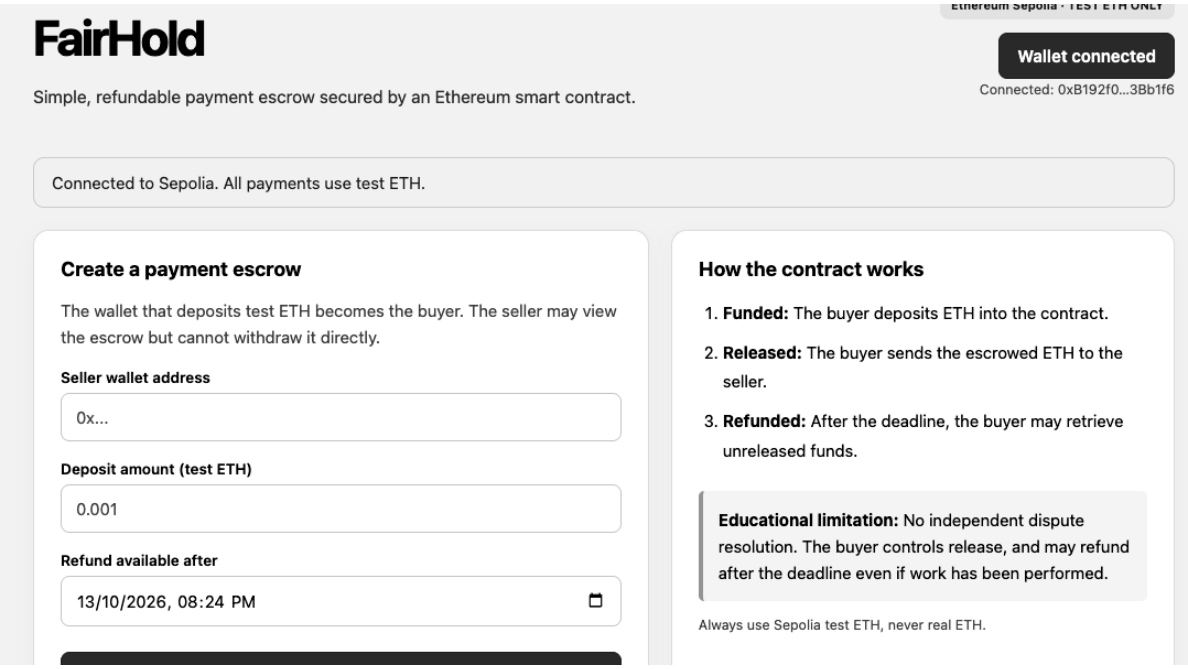


Figure 2. Capture of the FairHold web application during Sepolia testing.

9. Flask Backend Integration

The GET / route serves the interface; GET /api/config returns the configured contract address, deployment block and network. POST /api/validate checks the two Ethereum address formats, distinct buyer and seller addresses, positive finite ETH values, whole-wei precision and a bounded refund deadline. The response includes the exact wei amount for comparison with the browser-side transaction value.

The optional GET /api/verify/<tx_hash> endpoint can query a Sepolia JSON-RPC provider for a transaction and receipt, checking whether it was mined, succeeded and targeted the configured contract. This endpoint requires a separately configured SEPOLIA_RPC_URL; successful end-to-end use was not demonstrated in the collected evidence.

Final settlement decisions remain on-chain. Backend validation serves as a preflight usability mechanism, while the smart contract must independently reject invalid or unauthorized transactions.

10. Functional Testing: Sepolia Transactions

Two independent escrow scenarios were exercised in the application. Each used 0.001 Sepolia test ETH. The interface reported confirmed transactions and displayed the state changes and event history shown below. Screenshots are application-level observations; they do not replace independent transaction-receipt inspection.

Scenario	Observed outcome
Escrow #1: deposit	Funds recorded with status Funded and EscrowCreated event.
Escrow #1: release	Buyer triggered release; status updated to Released with EscrowReleased event.
Escrow #2: deposit and refund	Buyer deposited and, after the deadline, requested refund; status updated to Refunded.
Transaction history	Created, Released and Refunded events displayed with Sepolia explorer links.

10.1 Transaction Evidence

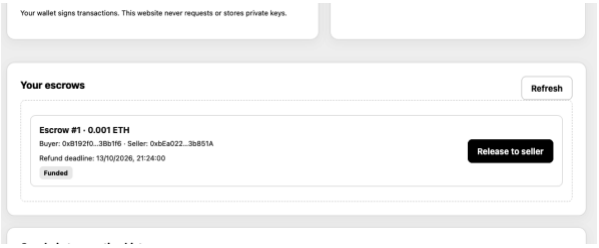


Figure 3. Escrow #1: Funded after deposit.

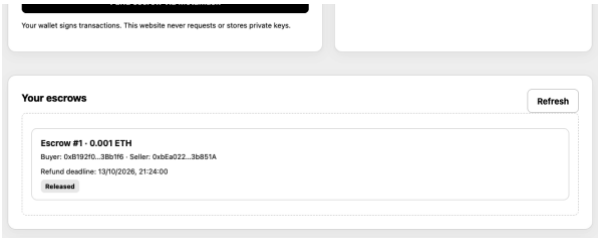


Figure 4. Escrow #1: Released after payment.

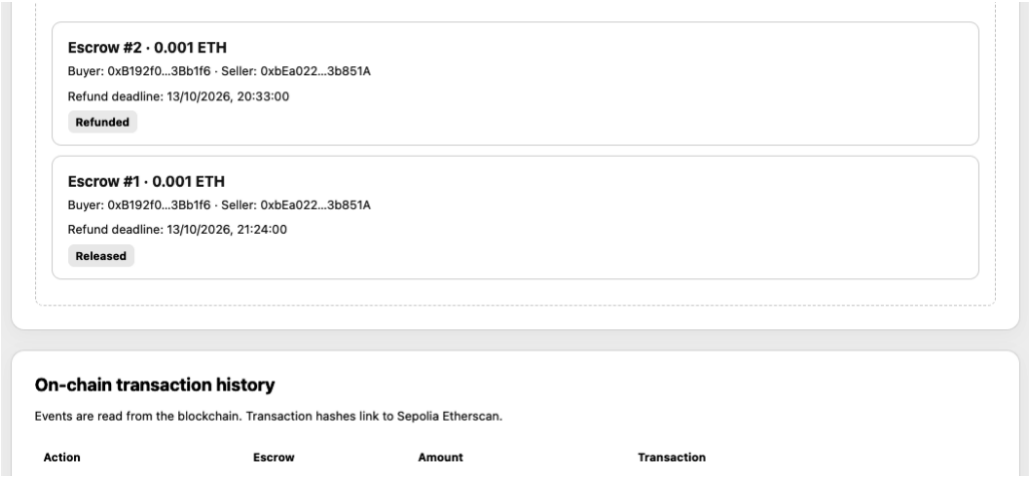


Figure 5. Escrow #2: Refunded status and event history.

11. Backend Tests and Validation

The test suite was executed in GitHub Codespaces with the command python -m unittest discover -s tests -v. The terminal reported seven passing tests in 0.145 seconds. These Flask test-client checks verify server-side validation responses, not smart-contract execution or wallet permissions.



Figure 6. Codespaces terminal evidence showing seven successful unittest cases.

Test case	Result
Valid input and 0.001 ETH to 10^15 wei conversion	Pass; HTTP 200
Seller equals buyer	Pass; rejected with HTTP 400
Zero deposit amount	Pass; rejected with HTTP 400
Sub-wei fractional amount	Pass; rejected with HTTP 400
Malformed seller address	Pass; rejected with HTTP 400
Expired refund deadline	Pass; rejected with HTTP 400
Invalid transaction hash	Pass; rejected with HTTP 400

11.1 Manual Input-Validation Checks

During interactive testing, the web interface was reported to reject an invalid seller address, a zero-value deposit and a past deadline. These frontend tests should not be confused with direct contract-level revert tests.

11.2 Tests Not Completed

Direct seller-wallet attempts to release or refund another participant's escrow; an early-refund revert through Remix; a duplicate settlement attempt; explicit MetaMask transaction rejection; and end-to-end optional Flask RPC verification were not demonstrated. These remain limitations of the test coverage.

12. Challenges Encountered

Initial deployment through Remix produced an insufficient-funds gas-estimation message because the deployment environment appeared connected to Ethereum Mainnet while test ETH was held on Sepolia. Reconnecting the deployment environment to the correct testnet resolved the mismatch. A separate attempt to connect the web interface with the seller MetaMask account was unsuccessful, leaving direct unauthorized-user testing incomplete.

13. Evaluation and Limitations

- Fairness and disputes: the buyer controls release, and no adjudicator, delivery oracle or seller-approved settlement is implemented.
- Production security: there is no external audit, formal verification, fuzz-test campaign or comprehensive contract-access testing.
- Operational scope: the demonstration runs using developer-hosted Flask and test ETH, without production monitoring or availability guarantees.
- Privacy: wallet addresses, transaction values and event data are public; these may disclose financial relationships.
- Usability and performance: wallet account switching was not fully validated; gas usage, transaction latency and throughput were not systematically benchmarked.

14. Future Improvements

A more balanced financial design could introduce mutual acceptance of milestones, seller acknowledgement, partial releases, a structured dispute window and optional independent arbitration. Technical development should include Solidity unit and fuzz tests, explicit negative access-control cases, a contract audit, RPC event pagination, reliable account-change handling and empirical gas and latency measurements.

15. Conclusion

FairHold demonstrates a functional testnet escrow DApp integrating Solidity custody, MetaMask transaction signing, a Flask validation backend and an on-chain event history. The two observed escrow workflows show successful buyer-authorized release and post-deadline refund; seven backend validation tests also passed. Its value as a teaching prototype lies in making settlement rules explicit and observable. The buyer-biased trust model and incomplete contract-security testing preclude any claim that the prototype is production-ready.

16. Deployment and References

Ethereum network: Sepolia (chain ID 11155111). Recorded contract address: 0xb422a2a6e0c6416141217339ca636c9e7739f96d. Reported deployment block: 11,905,977. These identifiers should be cross-checked against the successful deployment receipt before formal submission.

Project source: https://github.com/maxinekoh/SC6113-Individual-DApps

SC6113 Individual Assignment, “Financial Decentralized Application (DApp),” course brief, Trimester 1.

Solidity documentation: https://docs.soliditylang.org/; ethers.js v6 documentation: https://docs.ethers.org/v6/.

Flask documentation: https://flask.palletsprojects.com/; Sepolia Etherscan: https://sepolia.etherscan.io/.